



2026 미래에셋증권 AI Festival Technical Session

이우람 수석매니저

미래에셋증권 AI Engineering 본부

01

Ontology란?

- 1-1. Ontology ≠ Knowledge Graph
- 1-2. 왜 Ontology가 중요한가?
- 1-3. Ontology는 어떻게 표현되는가?
- 1-4. 무엇이 추론을 가능하게 하는가?
- 1-5. Ontology는 AI의 Semantic Layer

02

Knowledge Graph 맛보기

- 2-1. 지식은 왜 Graph가 되었을까?
- 2-2. Knowledge Graph란 무엇인가?
- 2-3. Manual Age → Generative Age
- 2-4. Graph는 관계를 따라간다

03

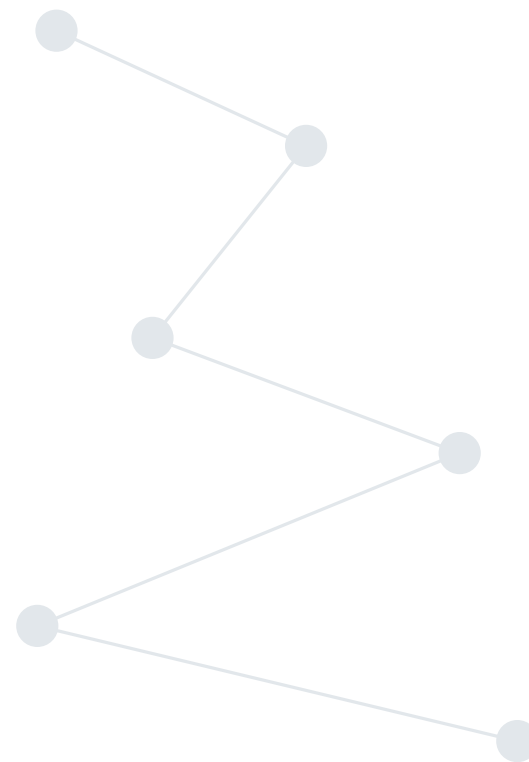
지식을 찾는 방법 : Retrieval

- 3-1. RAG와 Vector Retrieval
- 3-2. 하나의 검색법으로는 부족하다
- 3-3. Federated Query Architecture
- 3-4. Summary

01

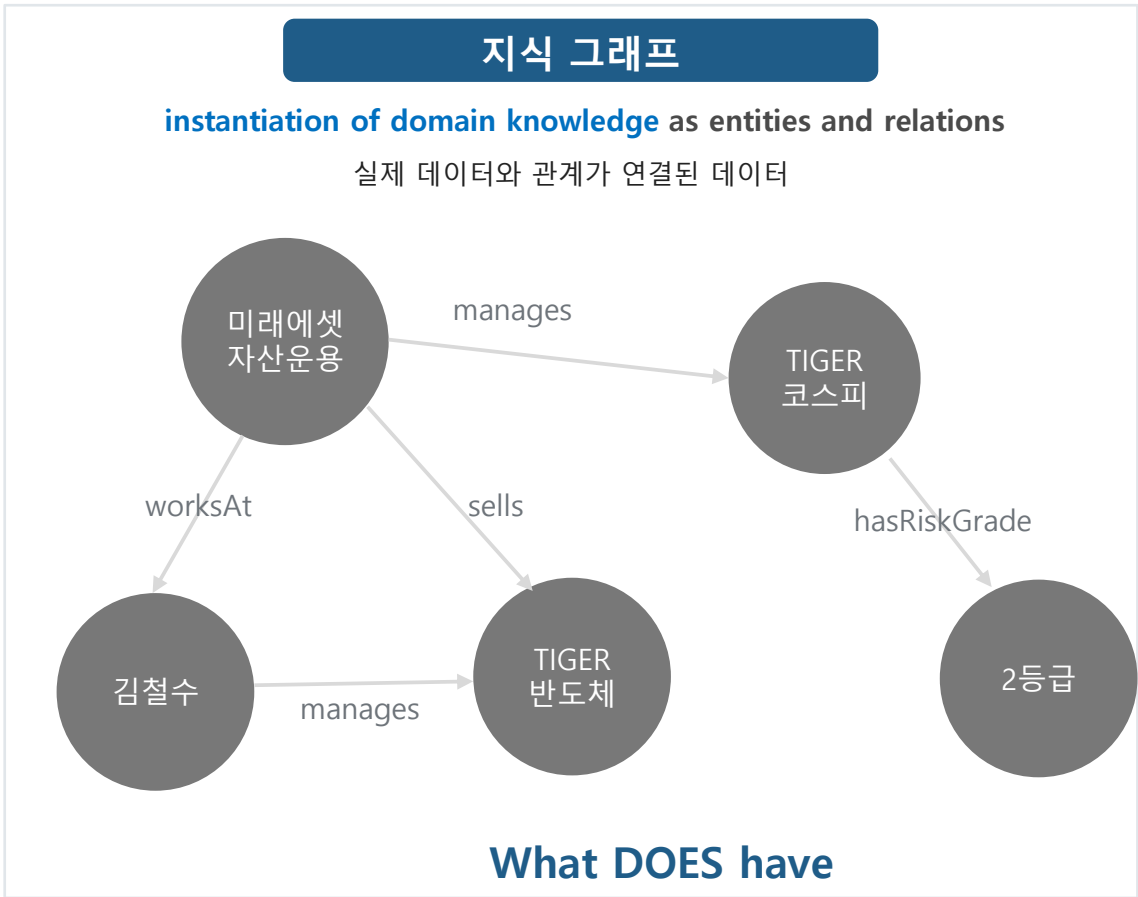
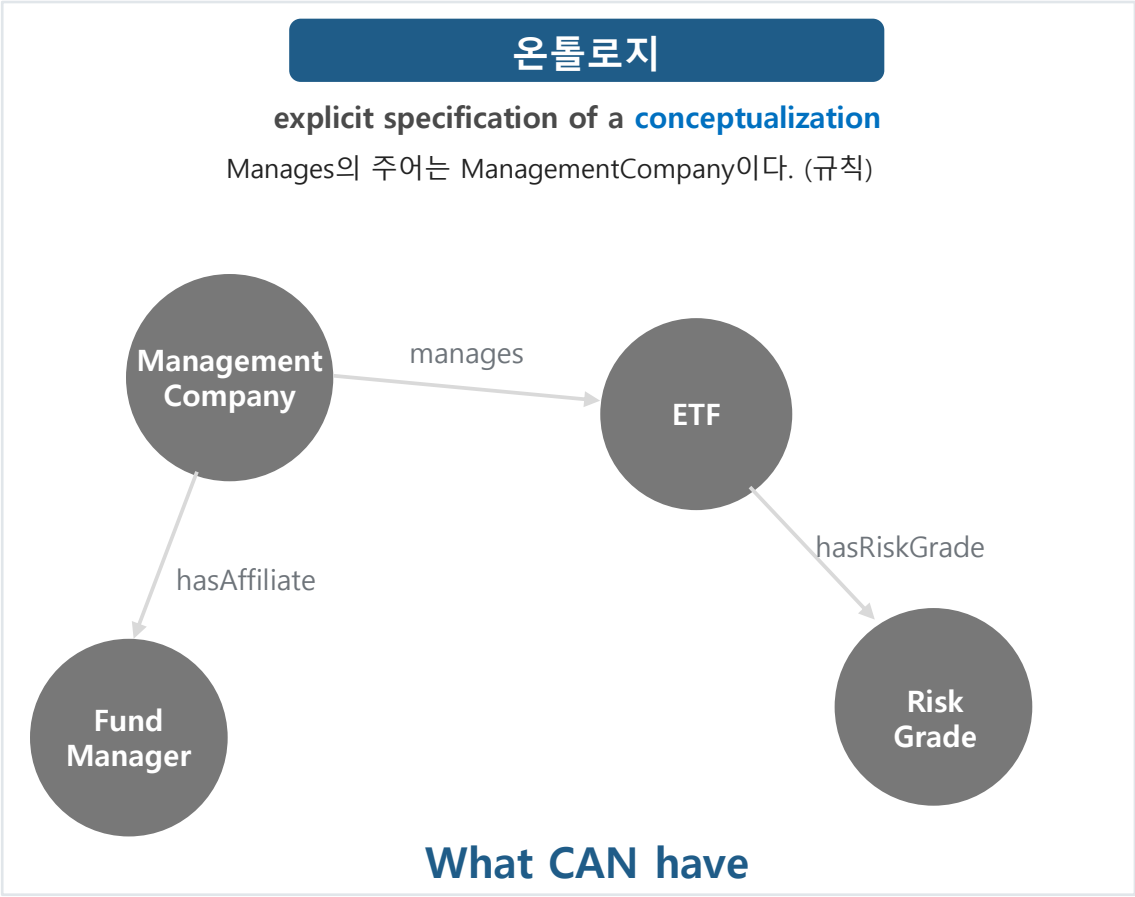
Ontology란?

AI가 데이터를 이해하려면, 먼저 세상의 의미(Semantics)가 정의되어야 한다.



01-1. Ontology ≠ Knowledge Graph

온톨로지는 “무엇이 가능해야 하는가”를 정의하고, 지식 그래프는 “실제로 무엇이 있는가”를 저장한다.



핵심: Ontology는 TBox(개념/규칙), Knowledge Graph는 ABox(실제 인스턴스)에 가깝다.

01-2. 왜 AI 시대에는 Ontology가 중요한가?

온톨로지는 모든 문제의 정답은 아니지만, 복잡한 지식을 AI가 이해하도록 돕는 전략적 토대가 될 수 있다.

용어 표준화

부서마다 다른 용어를
하나의 의미로 통일

예시

운용사 · 자산운용사 · AMC
→ ManagementCompany

지능형 추론

관계와 규칙을 따라
숨겨진 연결을 도출

예시

운용사 → 상품 방향 저장해도
상품 → 운용사 질문에 응답

데이터 무결성

위험등급·운용사 같은
비즈니스 규칙 검증

예시

riskGrade = 99 ×
허용: 1~5등급

AI 신뢰성

LLM 답변을 회사의
지식 그래프에 연결

예시

질문 → 근거 그래프
→ 답변 생성

모든 곳에 만들 필요는 없다. 정밀도와 의미 연결이 중요한 도메인부터 점진적으로 적용한다.

01-3. Ontology는 어떻게 표현되는가?

URI로 이름을 붙이고, RDF로 사실을 쓰고, RDFS/OWL로 의미와 제약을 정의한다.

Identifier	URI / IRI / Prefix 개념마다 세상에서 유일한 이름을 붙인다
Data Model	RDF 주어-서술어-목적어(Triple)로 사실을 쓴다
Schema Vocabulary	RDFS 클래스와 속성의 종류를 정의한다
Ontology Language	OWL 관계의 제약과 논리 규칙을 선언한다
Validation	SHACL / ShEx 데이터가 규칙을 지키는지 검사한다

아래에서 위로 쌓인다 — 아래 층은 “무엇으로 쓰는가”, 위 층은 “무엇이 맞는가”

직렬화: 같은 의미를 어떤 파일로 쓸까?

Turtle (.ttl)

사람이 읽기 가장 쉬움. 실무 기본값.
이 세션의 예제도 모두 Turtle.

RDF/XML

가장 오래된 표준 형식.
시스템 간 교환에 사용.

JSON-LD

웹-API 친화적.
JSON 그대로 다룰 수 있음.

세 형식은 표기만 다를 뿐
담고 있는 의미는 완전히 같다.

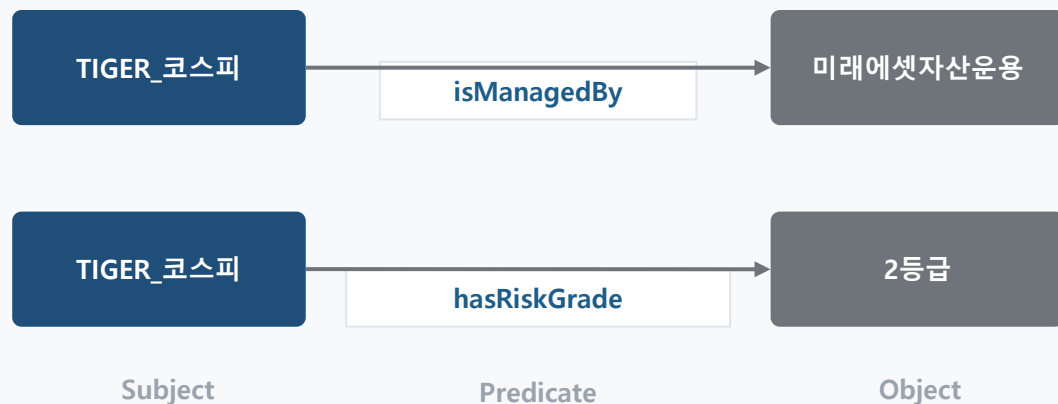
핵심: 온톨로지는 “파일 형식”이 아니라 “의미의 층”이다. 어떤 형식으로 저장해도 규칙은 그대로다.

01-3 (1) 한 문장을 Triple로 옮겨보기

자연어 한 문장은 여러 개의 Triple로 쪼개지고, 각 Triple은 규칙으로 검사된다.

“TIGER 코스피는 미래에셋자산운용이 운용하는 ETF이며, 위험등급은 2등급이다.”

① 사실로 쪼갬다 — RDF Triple



② 규칙으로 검사한다 — OWL / SHACL

riskGrade는 1~5등급만 존재한다

ETF는 isManagedBy를 정확히 1개 가져야 한다

규칙에 비추어 데이터를 검사하면

✓ TIGER_코스피 isManagedBy 미래에셋자산운용

✗ TIGER_코스피 hasRiskGrade 99등급

→ 99등급은 규칙 위반. 저장되기 전에 걸러진다.

핵심: 사람이 읽는 한 문장이, 기계가 검사할 수 있는 사실 단위로 바뀐다.

01-3 (2) TTL 파일로 보는 Ontology

TTL은 Ontology의 Class, Property, Constraint를 기계가 검증 가능한 **코드로 선언한 설계도**입니다.

etf_base_ontology.ttl — 핵심 구조

```
etf:ETF a owl:Class ;  
  rdfs:label "상장지수펀드(ETF)"@ko ;  
  rdfs:subClassOf fibo:FinancialInstrument ;  
  owl:disjointWith etf:ETN .
```

```
etf:hasStrategy a owl:ObjectProperty,  
  owl:FunctionalProperty ;  
  rdfs:domain etf:ETF ;  
  rdfs:range etf:Strategy .
```

```
etf-sh:ETFClassificationShape a sh:NodeShape ;  
  sh:targetClass etf:ETF ;  
  sh:property [  
    sh:path etf:hasStrategy ;  
    sh:minCount 1 ; sh:maxCount 1 ;  
    sh:in ( etf:Passive etf:Active )  
  ] .
```

코드가 말하는 3가지

Class

ETF는 금융상품이며
ETN과 명시적으로 구분

Property

hasStrategy는
ETF → Strategy 관계

Constraint

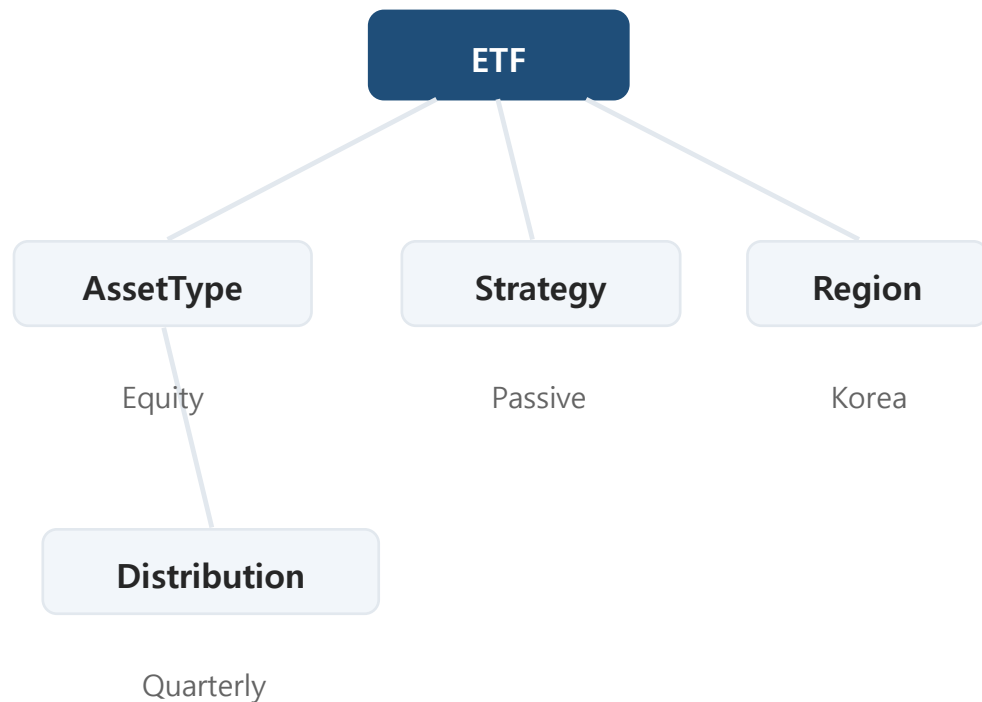
전략값은 Passive/Active 중
정확히 1개만 허용

**핵심: TTL은 “표 구조”가 아니라
의미와 규칙을 함께 담은 코드입니다.**

01-3 (3) Mermaid ERD로 읽는 ETF Ontology

핵심 Entity와 관계만 남겨 Ontology 구조를 직관적으로 보여준다.

핵심 Class 관계



관계 예시

```
ETF
├── hasAssetType → Equity
├── hasStrategy  → Passive
├── hasRegion    → Korea
└── hasDistribution → Quarterly
```

실제 Ontology는 수백~수천 개의 Entity로 구성되지만, 사람이 이해하기 위해서는 핵심 관계만 시각화한다.

01-4. 무엇이 추론을 가능하게 하는가?

공리(Axiom) + 논리 규칙 = 새로운 사실이 자동으로 생긴다. 2개를 넣으면 5개가 된다.

① 공리 — OWL로 선언한 규칙

ETF는 FinancialProduct의 하위 클래스다
isManagedBy의 목적어는 ManagementCompany다
isManagedBy의 역(inverse)관계는 manages다

② 논리 규칙 — 추론 엔진이 아는 상식

Type 상속: x 가 A 이고, A 가 B 의 하위면 $\rightarrow x$ 는 B 다
Domain 추론: $x P y$ 이고 P 의 domain이 D 면 $\rightarrow x$ 는 D 다
Range 추론: $x P y$ 이고 P 의 range가 R 면 $\rightarrow y$ 는 R 다

내가 직접 쓴 사실 — 2개

```
etf:TIGER_코스피 a etf:ETF .  
etf:TIGER_코스피 etf:isManagedBy etf:MiraeAsset .
```

추론 엔진이 자동으로 만든 사실 — 3개

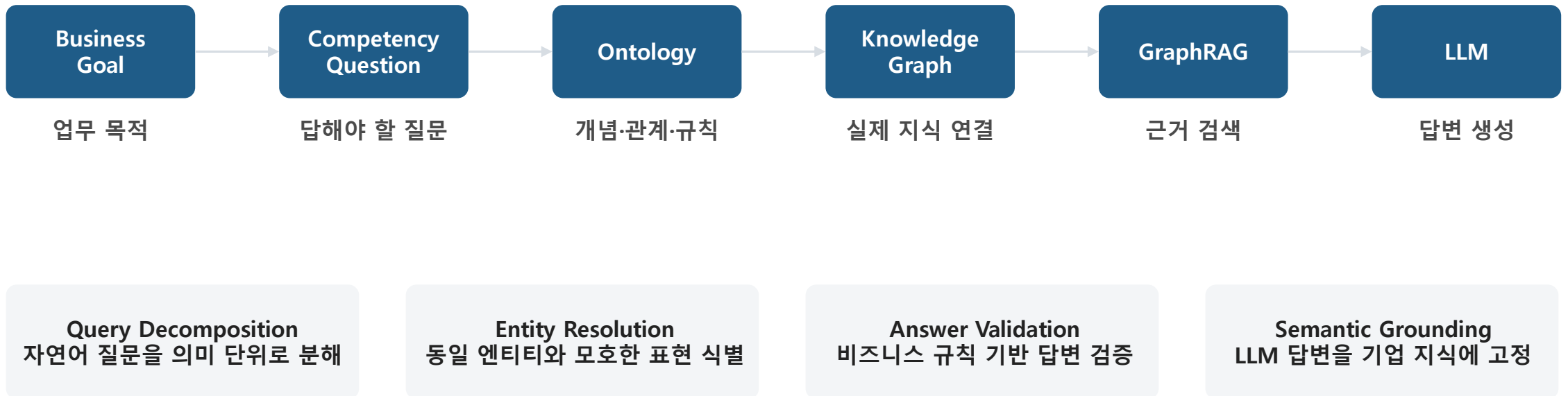
```
etf:TIGER_코스피 a etf:FinancialProduct .  
etf:MiraeAsset a etf:ManagementCompany .  
etf:MiraeAsset etf:manages etf:TIGER_코스피 .
```

명시 트리플 2개 $\rightarrow$ 추론 트리플 3개 추가 = 총 5개

핵심: 온톨로지는 용어 사전이 아니다. 적어두지 않은 사실까지 규칙으로 스스로 만들어낸다.

01-5. Ontology는 AI의 Semantic Layer이다

LLM은 언어를 이해하지만, 기업 AI가 안정적으로 답하려면 데이터의 의미·규칙·경로를 해석하는 층이 필요하다.

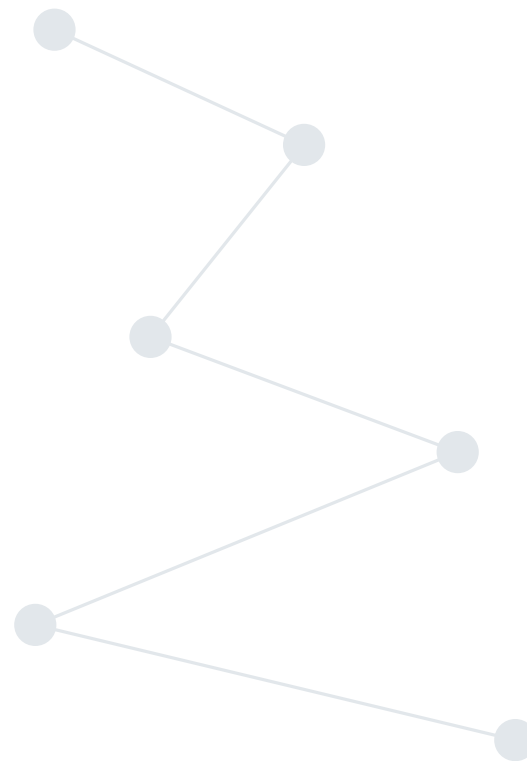


“LLM은 언어를 이해하고, Ontology는 의미(Semantics)를 이해한다.”

02

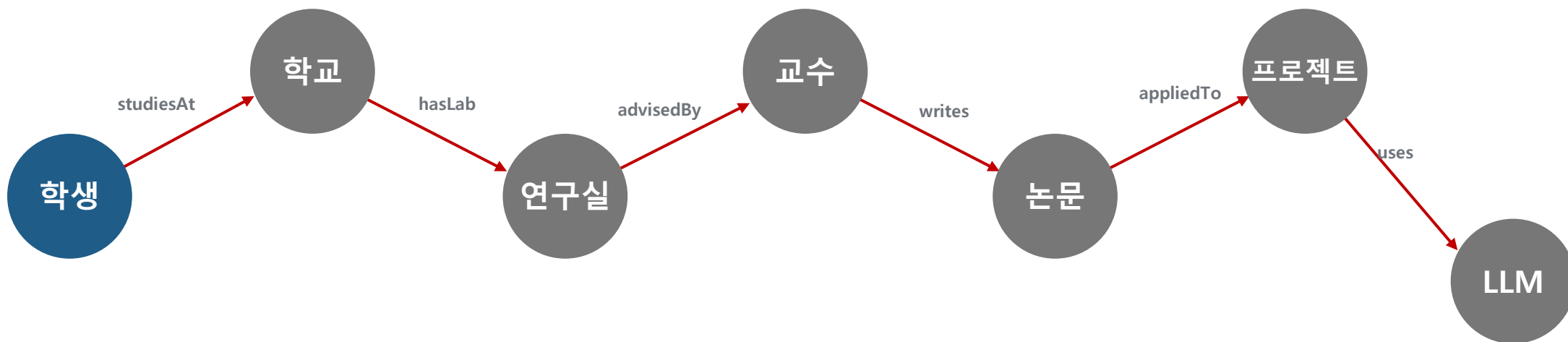
Knowledge Graph 맛보기

개념을 정의했다면, 이제 실제 지식을 연결해야 한다.



02-1. 지식은 왜 Graph가 되었을까?

세상은 표 하나보다 연결망에 가깝다. 사람, 학교, 회사, 프로젝트, 논문은 모두 관계로 이어진다.

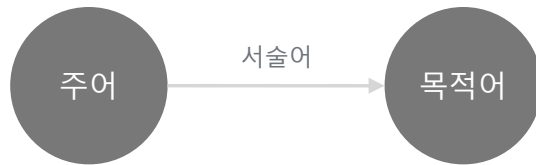


Knowledge Graph = Entity(개체) + Relation(관계) + 의미가 있는 연결

02-2. Knowledge Graph란 무엇인가?

컴퓨터가 이해할 수 있는 지식의 최소 단위는 Triple이고, Triple이 모이면 Graph가 된다.

Triple



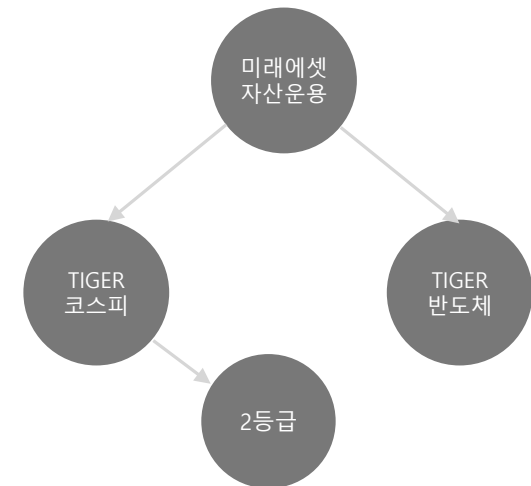
하나의 Triple = 하나의 사실

Example



“미래에셋자산운용이
TIGER 코스피를 운용한다”

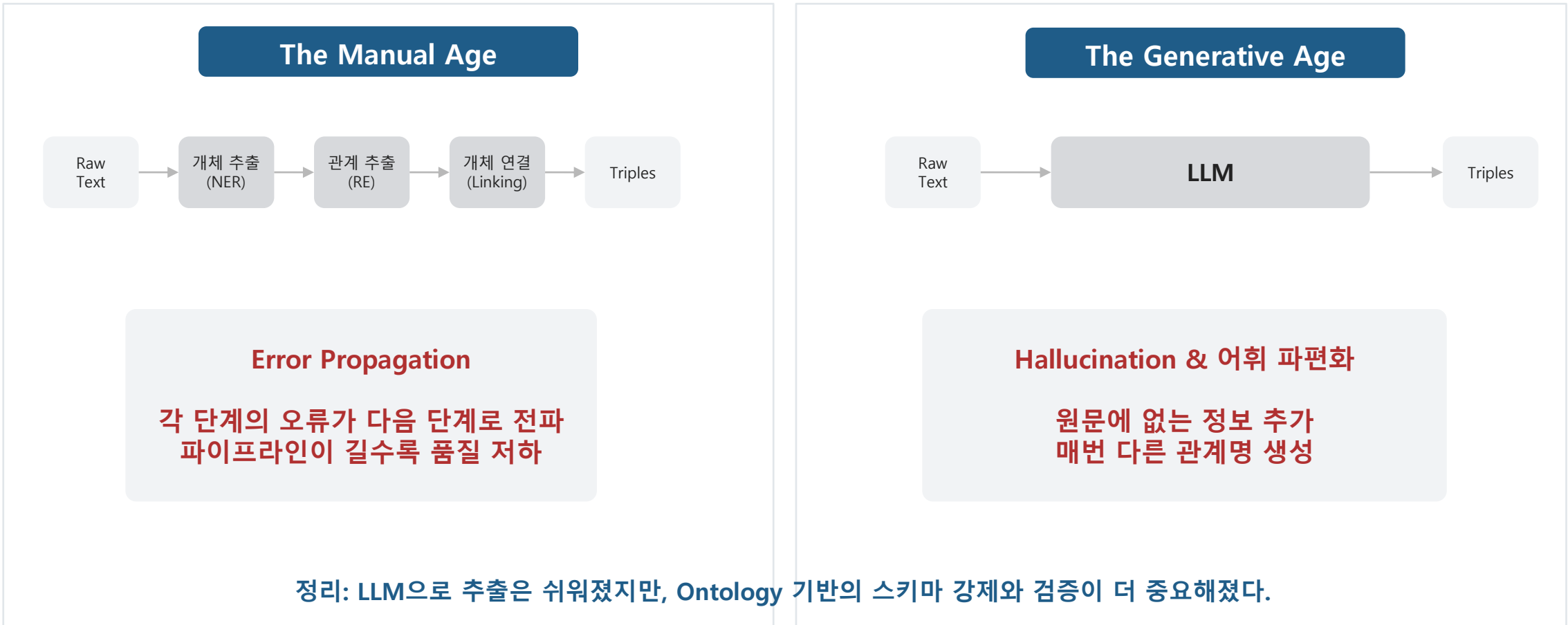
Graph



많은 사실을 연결하면
질문에 따라 탐색 가능

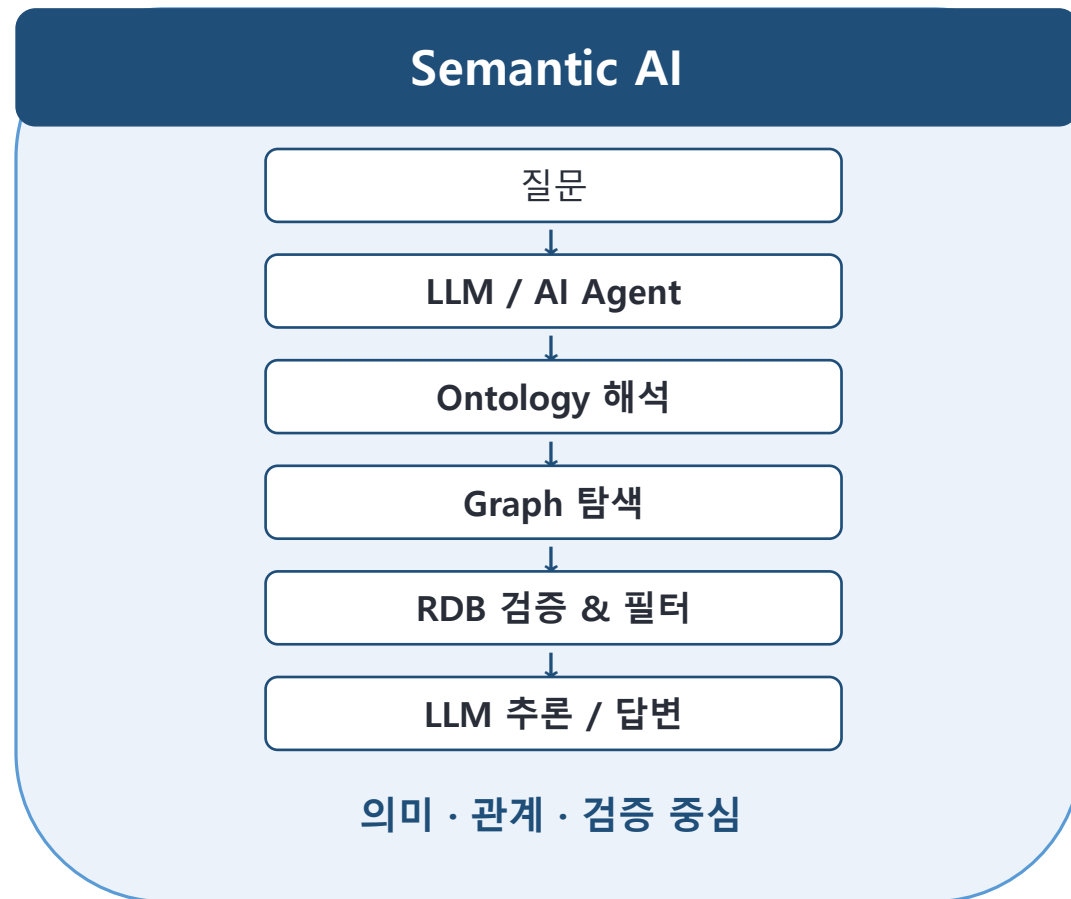
02-3. 지식 그래프 구축: Manual Age에서 Generative Age로

LLM은 트리플 추출을 단순화했지만, Hallucination과 어휘 파편화라는 새로운 문제를 만든다.



02-3 (1) 같은 LLM, 다른 지식 활용 방식

두 방식 모두 LLM을 사용하지만, LLM에 어떤 지식과 구조를 제공하는지가 다르다.



예시 질문 | 국내 배당형 ETF 중 분기배당이고 운용보수 0.1% 이하인 상품을 추천해줘.

핵심: 같은 LLM이라도, **문서 조각만 주는 방식**과 **Ontology · Graph · RDB 근거를 함께 주는 방식**은 답변 품질이 다르다.

02-4. Graph는 관계를 따라간다

Vector 검색은 비슷한 문서를 찾지만, Graph는 명시적인 관계 경로를 따라 답을 찾는다.

질문: “에코프로의 상장 자회사를 편입한 ETF의 운용사는?”



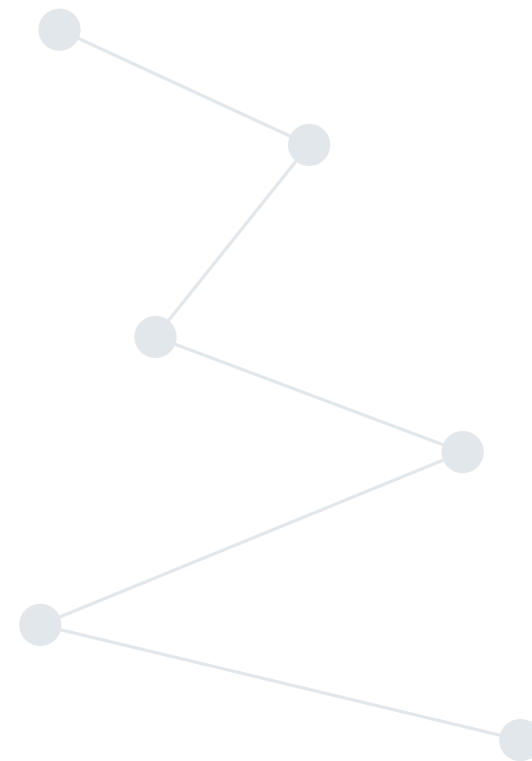
```
corp:에코프로 corp:hasSubsidiary ?sub  
?etf etf:hasConstituent ?sub → ?etf etf:managedBy ?am
```

핵심: “자회사 → 편입 → 운용사”처럼 여러 단계를 순회해야 하는 질문은 Graph가 강하다.

03

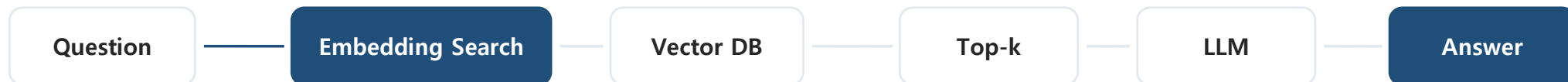
지식을 찾는 방법: Retrieval

연결된 지식은 "잘 찾을 수 있을 때" 비로소 답변이 된다.



03-1. RAG: 검색한 내용을 근거로 답한다

질문을 의미 벡터로 바꾸고, 가까운 문서를 찾아 LLM 답변의 근거로 사용한다.



Pitfall: Fragmented Retrieval

chunk #1

상품명: 미래에셋TIGER2차전지테마증권상장지수투자신탁
운용사: 미래에셋자산운용 · 기초지수: WISE 2차전지 테마 지수

chunk #2

[보수율 테이블] 판매수수료 -, 총보수 0.5%, 총보수·비용 0.55% ← 상품명 없음

chunk #3

[투자위험] 주식가격변동위험 · 추적오차 위험 ← 상품명 없음

질문: “TIGER 2차전지테마의 보수율과 추적오차 위험을 같이 알려줘”

상품명인 들어간 chunk #1만 잘 검색되고, 보수율/위험 정보가 있는 chunk #2/#3은 누락될 수 있다.

03-2. 하나의 검색법으로는 부족하다

복잡한 질문 하나에도 의미 유사도, 관계 순회, 숫자 필터가 동시에 필요하다.

Running Example: “에코프로의 상장 자회사를 편입한 ETF 중, 순자산 1,000억 원 이상이고 리서치 의견이 긍정적인 상품 추천”

Vector

의미 유사

“리서치 의견 긍정”

키워드 (BM25)

정확 매칭

“정확한 상품명”

Graph

관계 순회

“자회사 → 편입”

SQL

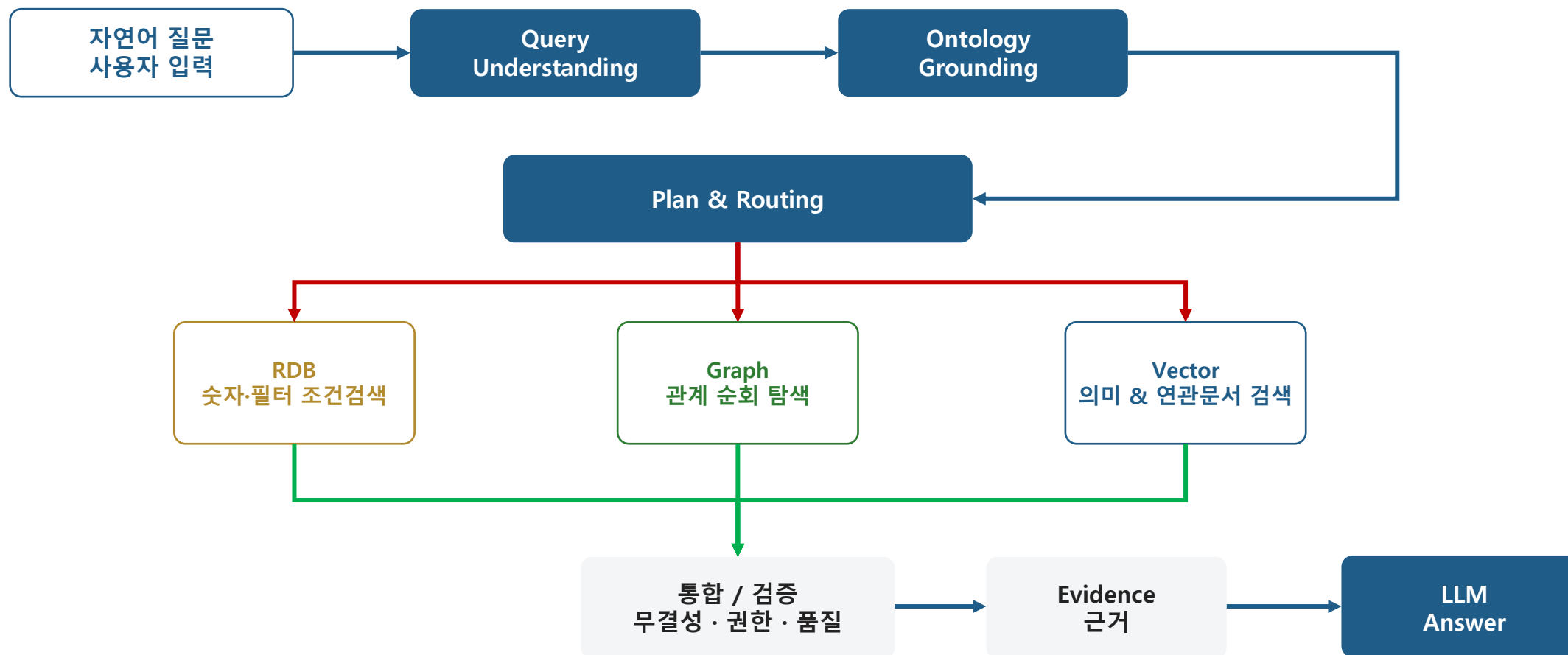
숫자 필터

“순자산 1,000억 이상”

결론 : Hybrid 검색은 “한 소스 내 결합”이고, Federated Query 는 “여러 소스 간 결합”이다.

03-3. Federated Query Architecture

자연어 질문을 의미 단위로 분해하고, 각 조각을 가장 적합한 소스로 보낸 뒤, 근거를 합쳐 답한다.



핵심 난제: Decomposition → Routing → Mapping 을 안정적으로 수행하는 메타데이터 계층이 필요하다.

03-3 (1) AI는 질문을 어떻게 분해할까?

복잡한 질문을 의미 단위로 쪼개고, Graph / RDB / Vector에 나누어 보낸다.

실제 질문

“국내 배당형 ETF 중 분기배당이고
운용보수가 낮은 상품 추천해줘.”

Query
Understanding

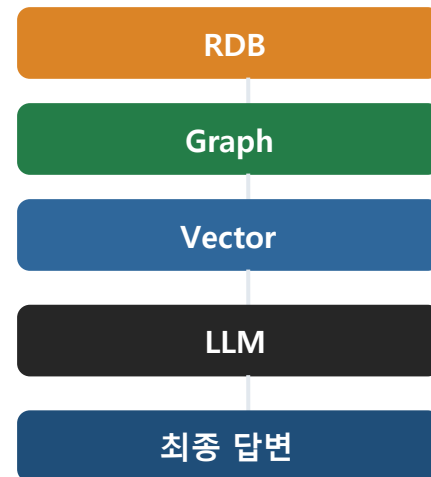
Ontology
Grounding



AI 답변 전략 수립

질문 요소	저장소
국내 ETF	Graph
배당형	Graph
분기배당	Graph
운용보수	RDB
“추천”	Vector

실제 실행 & 결과



핵심

Graph는 관계를 찾고, RDB는 숫자를 필터링하며,
Vector는 의미를 검색한다.

03-3 (2) 실험으로 확인한 것: 무엇이 검색 정확도를 만드나

국내 ETF 도메인 · 자체 벤치마크 72문항 · Semantic Layer 구성요소를 하나씩 더하며 측정

01 실행되는 SQL ≠ 맞는 답

SQL 실행 성공률 ~100%

실제 정답률 45%

→ 구문은 쉽고, 정답은 어렵다. 컬럼·값·조건을 틀리게 고른다.

02 온톨로지는 “설명”이 아니라 “실행”에

프롬프트에 설명으로 주입 +2.4pp

런타임 식별에 사용 +20pp

→ 온톨로지 = 문서가 아니라 런타임 통제 계층.

03 그래프는 RDB가 못 푸는 걸 푼다

RDB — ‘테마’ 컬럼 없음 0건

Graph — 이차전지 테마 ETF 36건

→ 관계·테마를 묻는 질의는 그래프의 몫.

04 한 가지 방법으론 천장이 있다

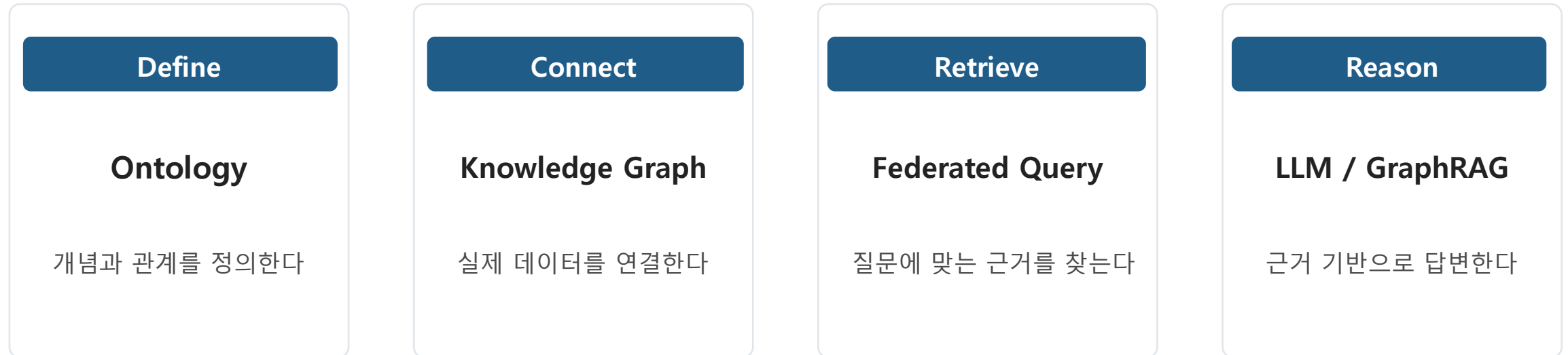
단일 검색기 최고치 ~66%

RDB + Graph 결합 가장 안정적

→ Federation은 선택이 아니라 필수.

단일 소스로는 안 된다 — 참고: NL2SQL도 스키마만 주면 8.3%, 비즈니스 의미까지 주면 78.3% (Painter et al., JAMIA Open 2025)

03-4. Summary: Define → Connect → Retrieve → Reason



AI가 데이터를 더 정확히 이해하려면, “문서 검색”을 넘어 “의미 기반 검색”으로 가야 한다.

Ontology는 설계도, Knowledge Graph는 연결된 데이터, Retrieval은 답을 찾는 엔진이다.

감 사 합 니 다

본 자료는 미래에셋증권이 제작한 것이며,
[2026 미래에셋증권 AI Festival] 기술관련 설명 목적으로 작성된 자료로써, 해당 목적 외 활용을 금합니다.

목적 외 활용으로 발생한 문제에 대해서 당사는 책임을 지지 않으며,
해당 자료를 다른 목적으로 전달/게시 한 경우 심사 상 불이익을 받을 수 있습니다.